

Simulación de Propagación de Incendio

Resolución paso a paso en Python

Clase de Programación

Profesor: Mateo Taito Stambuk

Email: mateo.taitos@edu.uai.cl

27 de Mayo, 2026

Objetivos de la Clase

- **Modelar** un sistema dinámico usando matrices
- **Practicar** listas anidadas, índices y mutabilidad
- **Desarrollar** funciones modulares y reutilizables
- **Comprender** la simulación por pasos discretos
- **Manejar** bordes y casos especiales en estructuras de datos

El Problema

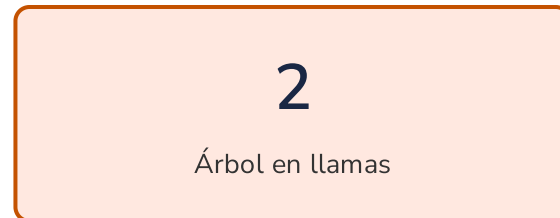
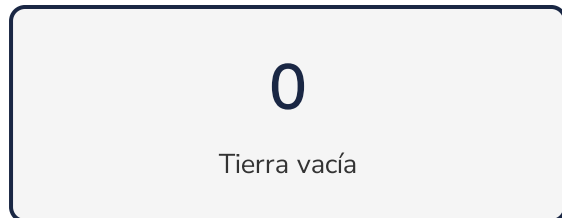
Simulación de Propagación de Incendio en un Bosque

Queremos modelar cómo se propaga un incendio a lo largo del tiempo.

Representación del bosque:

- Matriz de tamaño $N \times M$ (filas $\times$ columnas)
- Cada celda es una parcela de tierra con un estado

Estados posibles:

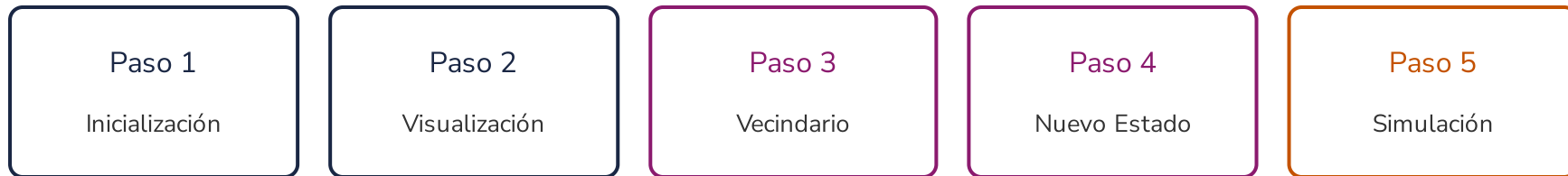


Reglas de Propagación

En cada turno, el estado del bosque cambia *simultáneamente*:

1. Árbol sano → En llamas
 - Si al menos un vecino adyacente ($\uparrow \downarrow \leftarrow \rightarrow$) está en llamas
2. Árbol en llamas → Tierra vacía
 - Se consume completamente en el siguiente turno
3. Tierra vacía → Tierra vacía
 - Permanece sin cambios

Arquitectura de la Solución



Haz clic para revelar cada paso →

1 Inicialización del Bosque

Paso 1: Inicialización

Crear la estructura de datos

Necesitamos una matriz $N \times M$ donde:

- La mayoría de las celdas son árboles sanos (1)
- Algunas celdas son focos iniciales de incendio (2)

Enfoque del código:

- Probabilidad del 90% → árbol sano
- Probabilidad del 10% → árbol en llamas

```
import random

filas = 10
columnas = 20
bosque = []
```

Paso 1: Código de Inicialización

```
for fila in range(filas):
    bosque.append([])
    for columna in range(columnas):
        if random.randint(0, 10) ≤ 9:
            bosque[fila].append(1) # Árbol sano
        else:
            bosque[fila].append(2) # Árbol en llamas
```

- Línea 1-2: Iteramos por cada fila y creamos una lista vacía
- Línea 3-6: Para cada columna, decidimos el estado aleatoriamente
- `random.randint(0, 10)` genera un número entre 0 y 10
- Si es ≤ 9 (90%): árbol sano (1)
- Si es > 9 (10%): árbol en llamas (2)

Paso 1: Diagrama de Flujo

[Ver diagrama de inicialización](#)

2

Visualización de la Matriz

Paso 2: Visualización

Hacer el bosque legible para humanos

Convertimos los valores numéricos en caracteres visuales:

.	T	X
Tierra vacía	Árbol sano	Árbol en llamas

Requisito: Recorrer la matriz con ciclos `for` anidados

Paso 2: Código de Visualización

```
bosque_visual = []  
for fila in range(filas):  
    bosque_visual.append([])  
    for columna in range(columnas):  
        if bosque[fila][columna] == 0:  
            bosque_visual[fila].append(".")  
        elif bosque[fila][columna] == 1:  
            bosque_visual[fila].append("T")  
        elif bosque[fila][columna] == 2:  
            bosque_visual[fila].append("X")
```

- Línea 1-3: Creamos una nueva matriz para los caracteres
- Línea 4-10: Recorremos cada celda y mapeamos el valor
- Usamos `if/elif` para seleccionar el carácter correcto

Paso 2: Imprimir el Bosque

```
for fila in range(filas):  
    for columna in range(columnas):  
        print(bosque_visual[fila][columna], end=" ")  
    print()
```

- Línea 1-3: Imprimimos cada carácter seguido de un espacio
- `end=" "` evita el salto de línea después de cada carácter
- Línea 4: `print()` sin argumentos genera el salto de línea al final de cada fila

Resultado visual:

```
T T T . X T T T T T  
T T T T T T T X T T  
. T T T T T T T T T
```

Paso 2: Diagrama de Flujo

[Ver diagrama de visualización](#)

3

Análisis de Vecindario

Paso 3: El Núcleo Lógico

¿Cómo saber si un árbol se incendia?

Para cada celda `(fila, columna)`, debemos revisar sus vecinos ortogonales:

```
      ↑ (fila-1, columna)
      |
← (fila, columna-1) • → (fila, columna+1)
      |
      ↓ (fila+1, columna)
```

Reto principal: Manejar los bordes de la matriz

- En `(0, 0)` no existe `fila - 1`
- En `(filas-1, columnas-1)` no existe `columna + 1`

Paso 3: Lógica de Quemado

```
for fila in range(filas):
    for columna in range(columnas):
        if bosque[fila][columna] == 2: # Árbol en llamas
            # Vecino arriba
            if fila - 1 ≥ 0 and bosque[fila - 1][columna] == 1:
                bosque[fila - 1][columna] = 3 # Estado intermedio
            # Vecino abajo
            if fila + 1 < filas and bosque[fila + 1][columna] == 1:
                bosque[fila + 1][columna] = 3
            # Vecino izquierda
            if columna - 1 ≥ 0 and bosque[fila][columna - 1] == 1:
                bosque[fila][columna - 1] = 3
            # Vecino derecha
            if columna + 1 < columnas and bosque[fila][columna + 1] == 1:
                bosque[fila][columna + 1] = 3
            # El árbol en llamas se consume
            bosque[fila][columna] = 0
            seguir_simulacion = 0 # Se quemó algo
```

Paso 3: Explicación Detallada

- Línea 1-3: Recorremos toda la matriz buscando árboles en llamas (2)
- Estado intermedio 3 : Marcamos los árboles que se quemarán en esta iteración
 - Esto evita que un árbol queme a sus vecinos **instantáneamente** en la misma iteración
- Línea 5, 9, 12, 15: Cada vecino verifica que esté dentro de los límites
 - $\text{fila} - 1 \geq 0$ → existe fila superior
 - $\text{fila} + 1 < \text{filas}$ → existe fila inferior
 - $\text{columna} - 1 \geq 0$ → existe columna izquierda
 - $\text{columna} + 1 < \text{columnas}$ → existe columna derecha
- Línea 18: El árbol en llamas se convierte en tierra vacía (0)
- Línea 19: Flag para continuar la simulación

Paso 3: Manejo de Bordes

¿Por qué es importante?

Sin verificación de bordes:

`bosque[-1][columna]` → Lee la ÚLTIMA fila (¡error lógico!)

`bosque[filas][columna]` → `IndexError` (¡crash!)

Solución aplicada:

Dirección	Condición	Significado
Arriba	<code>fila - 1 ≥ 0</code>	No estamos en la primera fila
Abajo	<code>fila + 1 < filas</code>	No estamos en la última fila
Izquierda	<code>columna - 1 ≥ 0</code>	No estamos en la primera columna
Derecha	<code>columna + 1 < columnas</code>	No estamos en la última columna

Paso 3: Conversión de Estado Intermedio

```
for fila in range(filas):  
    for columna in range(columnas):  
        if bosque[fila][columna] == 3:  
            bosque[fila][columna] = 2 # Ahora quema en la siguiente iteración
```

- Después de quemar todos los vecinos, convertimos los 3 en 2
- Los árboles que eran 3 ahora son focos de incendio para el siguiente turno
- Esto garantiza que la propagación sea paso a paso, no instantánea

Paso 3: Diagrama de Flujo

[Ver diagrama de vecindario](#)

4

Generación del Nuevo Estado

Paso 4: Transición de Estados

El estado intermedio 3 es clave

Problema: Si quemamos directamente $1 \rightarrow 2$, un árbol podría propagar fuego a todos sus vecinos en una sola iteración.

Solución: Usamos un estado intermedio 3

Iteración actual:

$1 \rightarrow 3$ (se quemó, pero aún no quema)

$2 \rightarrow \emptyset$ (se consumió)

Entre iteraciones:

$3 \rightarrow 2$ (ahora es foco de incendio)

Flujo completo de estados:

1 (sano) $\rightarrow 3$ (quemándose) $\rightarrow 2$ (en llamas) $\rightarrow \emptyset$ (cenizas)

Paso 4: Código Completo de Transición

```
seguir_simulacion = 1 # Asumo que no se quemará nada

for fila in range(filas):
    for columna in range(columnas):
        if bosque[fila][columna] == 2:
            if fila - 1 ≥ 0 and bosque[fila - 1][columna] == 1:
                bosque[fila - 1][columna] = 3
            if fila + 1 < filas and bosque[fila + 1][columna] == 1:
                bosque[fila + 1][columna] = 3
            if columna - 1 ≥ 0 and bosque[fila][columna - 1] == 1:
                bosque[fila][columna - 1] = 3
            if columna + 1 < columnas and bosque[fila][columna + 1] == 1:
                bosque[fila][columna + 1] = 3
            bosque[fila][columna] = 0
            seguir_simulacion = 0

for fila in range(filas):
    for columna in range(columnas):
        if bosque[fila][columna] == 3:
            bosque[fila][columna] = 2
```


Paso 4: Diagrama de Flujo

[Ver diagrama de nuevo estado](#)

5

El Ciclo de Simulación

Paso 5: Programa Principal

Estructura del ciclo principal

```
seguir_simulacion = 0
numero_iteracion = 0
while seguir_simulacion == 0:
    seguir_simulacion = 1

    # Lógica de quemado (Paso 3 y 4)
    # ...

    # Imprimir estado actual
    # ...

    numero_iteracion += 1
```

- Flag `seguir_simulacion` : Controla cuándo detener la simulación
- Contador `numero_iteracion` : Registra cuántos turnos se ejecutaron
- Condición del while: Se detiene cuando no se quema nada más

Paso 5: Flujo de Cada Iteración

En cada turno del ciclo `while` :

1. Asumir que no se quemará nada → `seguir_simulacion = 1`
2. Recorrer el bosque buscando árboles en llamas
3. Quemar vecinos marcándolos como `3`
4. Consumir árboles en llamas → `2 → 0`
5. Convertir estados intermedios → `3 → 2`
6. Imprimir el estado actual del bosque
7. Incrementar contador de iteraciones
8. Verificar flag: Si `seguir_simulacion == 0`, repetir

La simulación termina cuando:

- No quedan árboles en llamas
- O no hay árboles sanos adyacentes al fuego

Paso 5: Código Completo del Ciclo

```
seguir_simulacion = 0
numero_iteracion = 0
while seguir_simulacion == 0:
    seguir_simulacion = 1

    for fila in range(filas):
        for columna in range(columnas):
            if bosque[fila][columna] == 2:
                if fila - 1 >= 0 and bosque[fila - 1][columna] == 1:
                    bosque[fila - 1][columna] = 3
                if fila + 1 < filas and bosque[fila + 1][columna] == 1:
                    bosque[fila + 1][columna] = 3
                if columna - 1 >= 0 and bosque[fila][columna - 1] == 1:
                    bosque[fila][columna - 1] = 3
                if columna + 1 < columnas and bosque[fila][columna + 1] == 1:
                    bosque[fila][columna + 1] = 3
                bosque[fila][columna] = 0
                seguir_simulacion = 0

    for fila in range(filas):
        for columna in range(columnas):
            if bosque[fila][columna] == 3:
                bosque[fila][columna] = 2
```

Paso 5: Diagrama de Flujo

[Ver diagrama de simulación](#)

6

Diagrama Integrador

Diagrama de Flujo Completo

Conexión de todas las secciones

[Ver diagrama completo](#)



Resumen

Conceptos Aprendidos

Estructuras de Datos

- Listas anidadas (matrices)
- Índices `matriz[filas][columna]`
- Mutabilidad de listas

Control de Flujo

- Ciclos `for` anidados
- Ciclo `while` con flag
- Condicionales `if/elif`

Diseño de Algoritmos

- Estado intermedio para sincronización
- Manejo de bordes en matrices
- Simulación por pasos discretos

Buenas Prácticas

- Funciones modulares
- Visualización para debugging
- Flags de control

¡Gracias!

¿Preguntas?

Simulación de Propagación de Incendio en Python